Contracting the System Specification

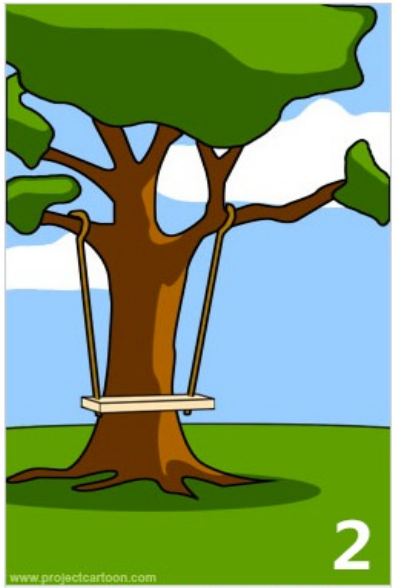
FINTECH 512
Software Engineering

Day 3

If behavior is not specified, the system will decide for you.

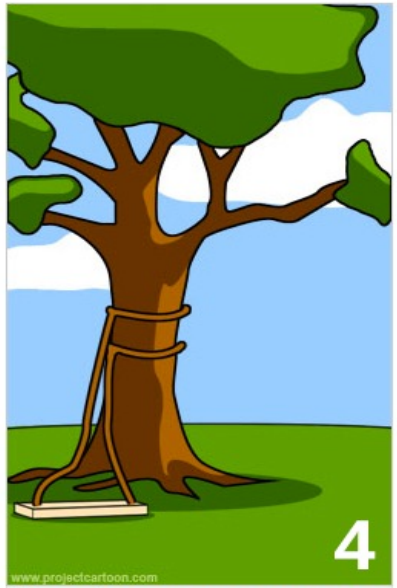
Duke

How the customer explained it



How the project leader understood it

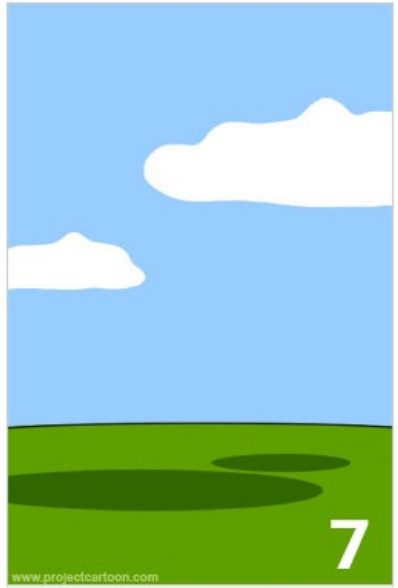
How the analyst designed it



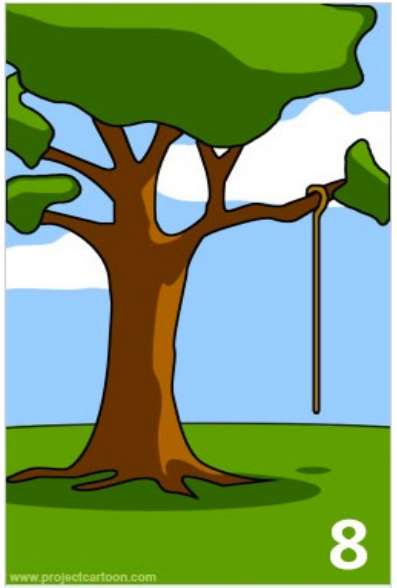
How the programmer wrote it

What the beta testers received

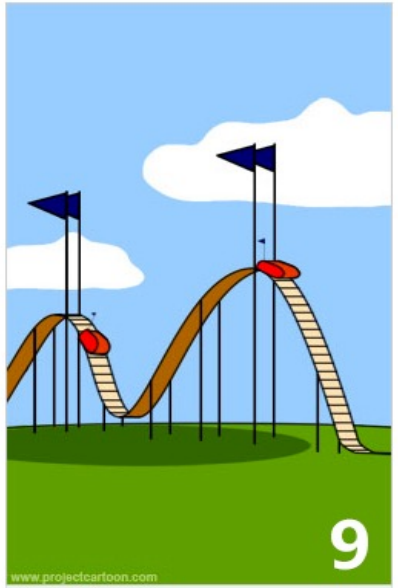
How the business consultant described it



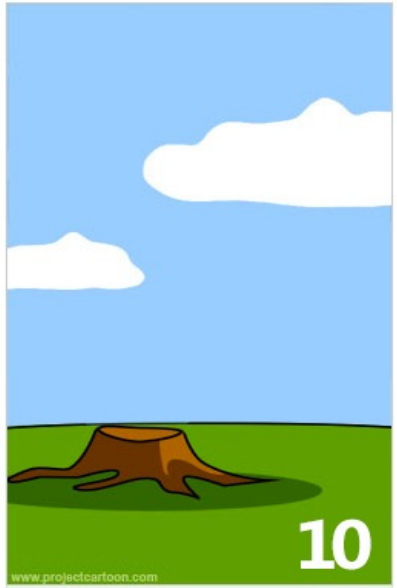
How the project was documented



What operations installed



How the customer was billed



How it was supported

What marketing advertised

What the customer really needed

Today's Goals

- Understand the issues in specifying requirements
- Understand why many bugs are requirement gaps
- Explain what it means to create a system specification
- Translate user stories into inputs, outputs, guarantees
- Identify edge cases and failure modes
- See how requirements, APIs, and tests connect

Where We Are So Far

- Lean Canvas defines **purpose**
- Requirements define **what must be true**
- User stories capture **intent**
- Acceptance criteria reduce ambiguity
- Tests make behavior precise

Today: we decide how the system responds to the world.

Duke

Why is this Hard?

Duke

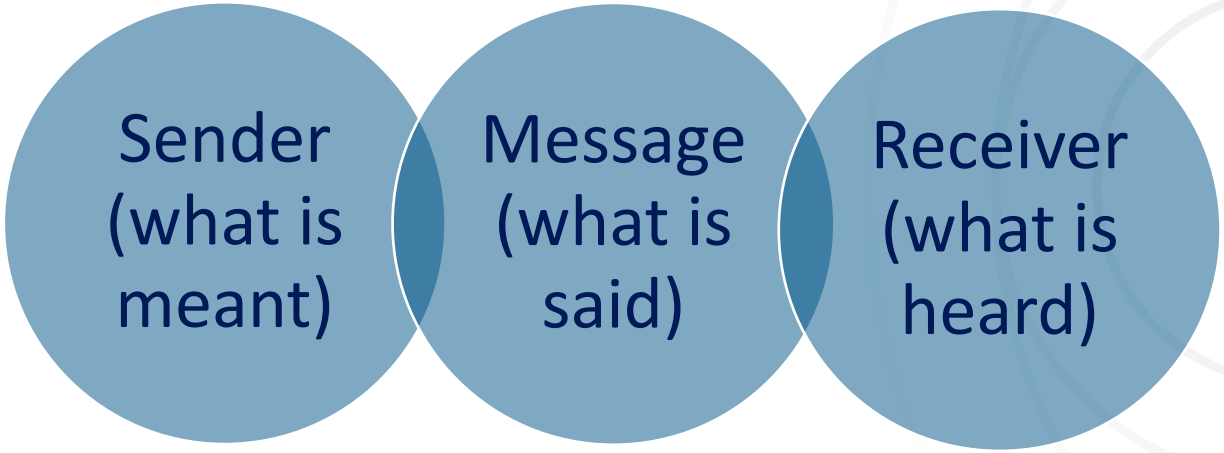
Communication

Image source: https://supereval.com/blog/open-communication-system/

Duke

Communication Issues

- Backgrounds: Cultural, gender, environmental
- Shared understanding
- Verbal, non-verbal, written



Duke

What is requirements engineering?

- *Knowledge acquisition* – how to capture relevant detail about a system?

  Is the knowledge complete and consistent?

- *Knowledge representation* – once captured, how do we express it most effectively?

  Express it for whom?

  Is it received consistently by different people?

Duke

Why Bugs Are Often Requirement Gaps

Where Do Bugs Really Come From?

- **Missing decisions** — behavior never defined
- **Ambiguous behavior** — multiple interpretations
- **Unspecified edge cases** — "what if" scenarios ignored
- **Conflicting assumptions** — teams disagree on intent

Most bugs are discovered requirements.

Duke

When Behavior Is Unspecified…

- Different developers make different assumptions
- Framework defaults decide behavior
- The system "chooses" an outcome

Undefined behavior is still behavior.

The system *always* does something. Silence = decision by default.

Duke

Finance Amplifies This Problem

- **Money is irreversible** — you can't undo a payment easily
- **Errors compound quickly** — small bugs multiply
- **Retries and duplicates are common** — network failures happen
- **Trust depends on correctness** — users expect perfect accuracy

Differences Among Concepts: Requirements, Specifications, Assumptions, and Design

Concept	Meaning
Requirements	What stakeholders want What should be true in the environment What problem must be solved
Specifications	What the system guarantees (must do)
Domain Assumptions	What we believe about the world
Design	How the system fulfills those guarantees (How will we build it?)

Confusing these causes failures.

Duke

What is a System Specification?

A contract that defines how a system responds to the world.

It defines:

- expected behavior
- allowed behavior
- forbidden behavior

Explicitly calls out:

- What is in scope
- What is out of scope
- What happens on failure

Duke

Contracting the System Specification

Making system behavior explicit, testable, and shared before implementation begins.

What is inside vs outside the system

What responsibilities the system guarantees

What inputs the system accepts

What outputs the system produces

What failures are allowed

Not code

Not frameworks

Not UI implementation

Not architecture diagrams

Duke

The Environment-Machine Boundary

Engineers must learn to ask:

- What exists **outside** the system?
- What does the system **control**?
- What does it **observe** but not control?
- Who is accountable if it fails?

Control
Observation
Responsibility

Why this matters today?

- APIs, cloud services, humans, and ML models all live at the boundary.
- Bugs often come from misplaced assumptions about control.

Duke

Characteristics of Requirements & Specifications

Measurable	We can tell if we met it Clear Objective
Descriptive	Of **what** the system should do, not how
Complete	No implicit understandings Everything written in the spec
Consistent	Don’t contradict each other

Duke

Fake vs Real Requirements

Fake Requirements	Real Requirements
Mention UI components or specific technology (constraints may still matter)	Describe world outcomes
Describe internal behavior	Allow multiple solutions
Assume the world behaves nicely	Make assumptions explicit
Collapse/confuse problem and solution	Express stakeholder value

Fake Requirement Diagnostic:

1. Does this describe the world or the software?
2. Can multiple designs satisfy this?
3. Does it mention UI, technology, or algorithms?
4. Does it depend on unstated assumptions?
5. Can we tell, objectively, if it's true in reality?

If changing the implementation breaks the "requirement", it was never a requirement.

Duke

Before / After Examples

Example 1: UI-Centered

- Bad: “The system shall display a red warning banner.”
- Good: “The user is clearly warned when data loss is imminent.”

Example 2: Technology-Centered

- Bad “The system shall store data in PostgreSQL.”
- Good: “Stored data persists across system restarts and failures.”

Example 3: Control Confusion

- Bad: “The user confirms their identity.”
- Good: “Only the legitimate account holder gains access.” *Assumption*: identity verification is reliable

Duke

From User Stories to Specification

Starting with a user story:

> As a user, I want to transfer money so that I pay someone.

Questions:

- What must the system do?
- What must never happen?

Duke

Use Cases

Describe how actors interact with the system to achieve a goal.

Helps to:

- Clarify steps
- Identify Alternatives
- Expose missing behavior

User Stories vs Use Cases

- User stories: intent and value
- Use cases: interaction sequences
- Acceptance criteria: correctness conditions

Duke

Use Case Name:	Transfer Money to another student		
Use Case ID:	TR 1		
Created by:	John Smith	Last Updated By:	John Doe
Date Created:	7 January 2020	Last Revision Date:	10 January 2020

Description	Allows a card user to transfer money from their personal account to another individual
Actors	Student card holder
Preconditions	1. Student has authenticated into the BluePay system 2. Student has funds in their account to be able to transfer to another student.
Postconditions	1. Sending student's balance is lowered by the amount of the transfer. 2. Receiving student's balance is increased by the amount of the transfer. 3. Appropriate audit trail established for non-repudiation.
Flow:	From the BluePay main screen, the user selects the transfer money icon. The system will then show a list of "friends" that the user has selected as their favorites. The user then selects a friend to transfer money to. The user then enters the amount to transfer. The system validates the user has sufficient and transfers the funds once the user has confirmed.
Alternate Flows	If the user does not have any "favorites" or the desired recipient is not in that list, the user can search for a new recipient. Insufficient funds. Duplicate requests. Invalid Inputs
Exceptions	If the transaction logs are unable to be made, the system rolls back the entire transaction and notifies the user of the cancellation. If the user has insufficient funds, the transaction is cancelled and the user notified.
Assumptions	
Notes and Issues	

Duke

Use Case Name:	Concise, results-oriented name for the use case. Reflect the tasks the user needs to be able to accomplish		
Use Case ID:	unique numeric identifier		
Created by:		Last Updated By:	
Date Created:		Last Revision Date:	

Description	Provide a brief description of the reason for and outcome of this use case, or a high-level description of the sequence of actions and the outcome of executing the use case.
Actors	An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks. Different actors often correspond to different user classes, or roles, identified from the customer community that will use the product. Name the actor(s) that will be performing this use case.
Preconditions	List any activities that must take place, or any conditions that must be true, before the use case can be started. Number each precondition
Postconditions	Describe the state of the system at the conclusion of the use case execution. Number each postcondition.
Flow:	Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions. This dialog sequence will ultimately lead to accomplishing the goal stated in the use case name and description.
Alternate Flows	Document other, legitimate usage scenarios that can take place within this use case separately in this section. State the alternative course, and describe any differences in the sequence of steps that take place.
Exceptions	Describe any anticipated error conditions that could occur during execution of the use case, and define how the system is to respond to those conditions. Also, describe how the system is to respond if the use case execution fails for some unanticipated reason.
Assumptions	List any assumptions that were made in the analysis that led to accepting this use case into the product description and writing the use case description.
Notes and Issues	List any additional comments about this use case or any remaining open issues or TBDs (To Be determineds) that must be resolved.

Duke

Inputs, Outputs, and Guarantees

Duke

Inputs: What Enters the System?

Inputs define what the system must handle.

Transfer Money Examples:

- Sender account
- Receiver account
- Amount
- Request identifier
- Authentication context

Duke

Outputs: What Can be Observed?

Outputs are the system's contract with the world.

Transfer Money Examples:

- Status: success / failure
- Error reason
- Confirmation code

Duke

State Changes & Guarantees

Guarantees are requirements the system commit to.

Transfer Money Examples:

- Balances updated
- Audit record written
- Exactly once processing

Duke

UI Sketches as Specification Tools

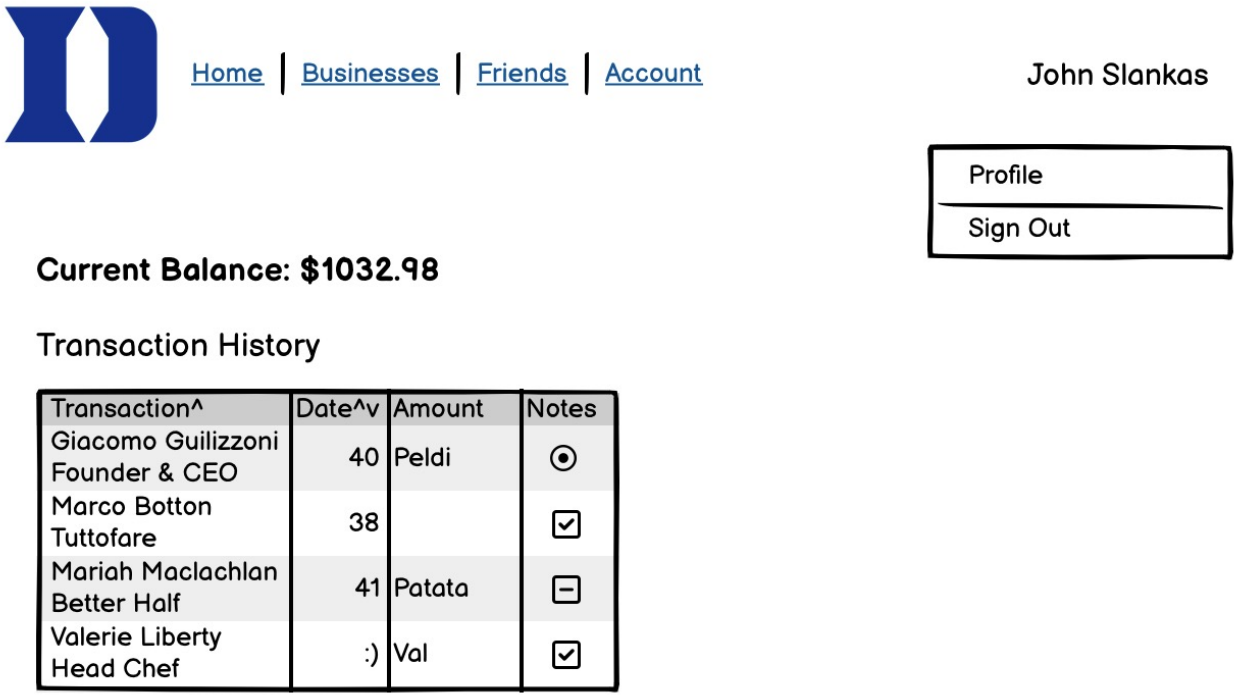
Duke

UI Sketching is an Engineering Activity

Mechanism to specify behavior, not a way to design aesthetics.

They help:

- Expose missing decisions and requirements
- Reveal edge cases
- Make behavior visible
- Force discussion of error states



Why UI Decisions Create System Bugs

UI choices determine :

- What actions are possible
- What feedback users receive
- When errors are visible
- How failures are handled

If behavior is not visible, users will assume something else happened.

Mockups & Prototypes

- Humans: better at recognizing whether a solution is correct than solving the problem from a blank page.
- Mock-ups/prototypes help explore uncertainty in the requirements.
  - Validate that we have the right requirements.
  - Elicit requirements at the “borders” of the system.
  - Assert feasibility of solution space.
  - Get feedback on a candidate solution.
- “I’ll know it when I see it”
- "People don't know what they want until you give them what they asked for."

Duke

UI Sketches Encode Requirements

A sketch answers:

- What happens after “Submit”?
- Can the user retry?
- Can they submit twice?
- What state is visible after failure

Every box and arrow implies a requirement.

Duke

Sketching Before Implementation

UI sketches are:

- fast
- cheap
- easy to change
- technology-independent

Sketches help contract behavior before code locks decisions in.

Interfaces as Behavioral Contracts

Duke

Systems Communicate Through Interfaces

- Systems receive **requests**
- Systems return **responses**
- Behavior must be observable

Interfaces define system boundaries.

Endpoints as Name Behavior

Examples:

- “Submit transfer”
- “Check transfer status”

Later:

- URLs
- HTTP verbs
- status codes

Why Interfaces Matter

- Force clarity about behavior
- Expose hidden assumptions
- Enable testing without implementation
- Enable integration with other systems

If you can't describe the interface, you don't understand the system.

Duke

Edge Cases & Failure Modes

Duke

Edge Cases are Not Rare

- **Invalid input** — malformed or out-of-range values
- **Boundary values** — zero, negative, maximum amounts
- **Duplicate requests** — same transfer submitted twice
- **Partial failures** — database writes but notification fails
- **Concurrency** — two transfers at the same time

Edge cases are not optional

FinTech Failure Modes

Common Scenarios

- Insufficient funds – sender does not have enough money
- Duplicate payment – same request processed twice
- Retry after timeout – use clicks submit multiple times
- Concurrent submissions – two transfers drain the same account

A Structured Way to Find Edge Cases

For each story ask:

- What if the input is **invalid**?
- What if it is **repeated**?
- What if it arrives **twice**?
- What if **part of the system fails**?
- What if it happens **at the same time** as something else?

Edge Cases → Acceptance Criteria → Tests

Each edge case:

- Becomes acceptance criteria
- Becomes a test

Tests don't find bugs – they enforce decisions.

Duke

Exercise: Contract the System Specification

Choose One user story:

- *As a user, I want to see my current account balance so that I know how much money I can spend.*
- *As a user, I want to view my recent transactions so that I can verify what has happened in my account.*
- *As a user, I want to know when a payment has failed so that I can take corrective action.*
- *As a user, I want to be notified when my account is accessed from a new device so that I can detect suspicious activity.*
- *As a user, I want to download my monthly account statement so that I can keep records for taxes or disputes.*

Group Task

- Identify: input and outputs
- Write: one short use case
- Sketch: one UI screen
- List: 3 edge cases

Rules

- No code
- No frameworks
- Behavior only

Duke

Connecting the Dots

Duke

Specification, UI, APIs, and Tests

- **Specification** defines behavior
- **UI** makes behavior visible
- **APIs** expose behavior to other parts / systems
- **Tests** enforce behavior

Different views of the same contract(decision).

Duke

Why This Comes Before Design

- Prevents architectural drift
- Reduces rework
- Makes design intention
- Enables safe evolution

Good design starts with clear behavior

Duke

Four Dark Corners of Requirements Engineering

- Requirements must refer to the real world
- Don't Describe the machine too early
- Make control explicit:
  - What the environment controls
  - What the machine controls
  - What is shared
- Domain knowledge enables refinement

Most real failures live in these corners.

Duke

AI-Assisted Ideation → Requirements: Best Practices

LLMs generate *candidate requirements*, not authoritative ones.

Human engineers remain responsible for correctness, completeness, and appropriateness.

1. Treat outputs as drafts
   - Assume incompleteness, ambiguity, and hidden assumptions
   - Never accept requirements “as-is”
   - Expect iteration and refinement
2. Prompt for structured requirement artifacts
   - User stories with acceptance criteria
   - Non-functional requirements checklist
   - Explicit assumptions and constraints
3. Enforce validation gates
   - Ambiguity checks (vague terms, subjective language)
   - Consistency checks (conflicting goals or requirements)
   - Traceability (every requirement maps to a goal or stakeholder need)

Quality is measured by:

- Testability
- Traceability
- Unambiguity
- Feasiblity

Duke

Prompt Template: Generate *Draft* Requirements

You are acting as a requirements analyst.

Create a DRAFT Product Requirements Document (PRD) for the following product idea.
Treat this as an initial proposal, not a final specification.

Include:
- Problem statement
- Target users and stakeholders
- Functional requirements (numbered)
- Non-functional requirements (performance, security, reliability, usability, scalability, availability, maintainability, compliance)
- Explicit assumptions
- Known risks and open questions

Product idea:
<insert idea here>

Duke

Prompt Template: Ambiguity & Quality Check

You are performing a requirements quality review.

Analyze the following requirements and identify:

1. Ambiguous or subjective terms (e.g., "fast", "easy", "robust")
2. Missing acceptance criteria
3. Requirements that are not testable
4. Hidden assumptions

For each issue, suggest a concrete rewrite.

Requirements:
<insert requirements here>

Duke

Prompt Template: Consistency & Conflict Check

Review the following requirements for internal consistency.

Identify:
- Conflicting requirements
- Requirements that imply incompatible design decisions
- Resource or constraint mismatches

Explain each conflict and propose a resolution strategy.

Requirements:
<insert requirements here>

Duke

Wrap-up & Review

What We Did Today:

- Contracted a system specification
- Used stories, use cases, and sketches
- Identified edge cases
- Connected specification to tests

Next Design:

- Organizing responsibilities
- Managing complexity
- Preparing systems to grow